MODELO DE MACHINE/ DEEP LEARNING QUE PERMITAM DISTINGUIR ENTRE TEXTO GERADO POR INTELIGÊNCIA ARTIFICIAL (E.G. CHATBOTS) E TEXTO ESCRITO POR SERES HUMANOS .

Alfredo Sócrates Elimila
alfredosocrateselimila@hotmail.com

Rita Tavita Carlos Elias
ritinhacarlos89@gmail.com

Cardoso da Silva Monteiro
cardosomonteiro7@gmail.com

Eunice Patrícia de Freitas Barros
929532160.eunice@gmail.com

Instituto Politécnicoda Universidade Katyavala Bwila, Angola

Resumo: Este artigo aborda o desafio contemporâneo de distinguir automaticamente entre textos em língua inglesa redigidos por seres humanos e conteúdos gerados sinteticamente por Grandes Modelos de Linguagem (LLMs). Seguindo as etapas formais da metodologia científica KDD (Knowledge Discovery in Databases), um corpus de dados multi-fonte foi seleccionado, pré-processado e transformado em matrizes numéricas. Desenvolveram-se duas abordagens computacionais competitivas: uma infraestrutura de baixo nível programada de raiz em NumPy, englobando uma baseline de Regressão Logística e uma Deep Neural Network (DNN) com regularização por Dropout; e um pipeline sequencial estruturado em PyTorch baseado em redes neuronais recorrentes bidirecionais com memória (Bi-LSTM) alimentadas por camadas de embeddings. Os modelos foram validados de forma não-enviesada, demonstrando excelente resiliência estatística em cenários de validação externa focados nas ciências naturais. Os resultados comprovam que a preservação da ordem cronológica e sintática das sequências textuais confere superioridade preditiva na captura das pegadas digitais deixadas por chatbots generativos.

Palavra-Chave: *Machine/Deep Learning, Classificação Binária, Treinamento, Implementação, Numpy, Pytorch, Deep Neural Networks, Baseline de Regressão Logística.*

Introdução

O crescimento exponencial de sistemas de Inteligência Artificial generativa, baseados na arquitectura de *Transformers*, transformou profundamente o ecossistema da produção de conteúdos digitais. Actualmente, aplicações de Inteligência Artificial (e.g., chatbots) são capazes de conceber ensaios académicos, relatórios técnicos, guiões e códigos de software com um nível de ligação gramatical, fluidez e consistência estilística que emula, com elevada precisão, o texto escrito por seres humanos. Esta evolução esgota as possibilidades de distinção visual macroscópica, criando a necessidade premente de conceber mecanismos automáticos e robustos capazes de auditar a autenticidade de documentos digitais e salvaguardar a integridade de repositórios contra o fenómeno de colapso de dados.

O presente artigo tem como objectivo central desenvolver modelos de *Machine/ Deep Learning* que permitam distinguir de forma automatizada entre texto gerado por modelos ou aplicações de Inteligência Artificial e texto escrito por seres humanos, executando uma tarefa de classificação binária sobre as etiquetas ai e human em documentos estruturados na língua inglesa. Para orientar o avanço científico do trabalho, a investigação teve como objectivos específicos:

- Rever a bibliografia e fundamentação teórica de classificadores textuais;
- Criar o repositório e estruturação inicial;
- Compilar, fundir e higienizar datasets multi-fonte em língua inglesa contendo respostas de humanos e de chatbots generativos;
- Conceber e treinar de raiz (usando exclusivamente a biblioteca NumPy) uma *baseline* de Regressão Logística e de uma *Deep Neural Network* (DNN);
- Implementar e optimizar uma arquitectura profunda recorrente bidireccional (*Bi-LSTM*) com *embeddings* em PyTorch;
- Desenvolver e ajustar hiperparâmetros dos modelos PyTorch;
- Preparar os pipelines de teste e 1ª Submissão dos modelos;
- Avaliar os modelos de forma competitiva face a conjuntos de validação externa.

Neste artigo, é apresentada uma revisão detalhada sobre engenharia de atributos aplicada à linguagem natural, combinando a extracção de matrizes estatísticas clássicas e o treinamento de uma baseline de Regressão Logística e redes profundas tabulares com pipelines recorrentes modernos.

1. Metodologia

Esta investigação segue a metodologia **KDD (*Knowledge Discovery in Databases*)**. Sendo um processo iterativo e sistemático de descoberta de conhecimento em bases de dados, a KDD organiza o fluxo de desenvolvimento em fases modulares que estendem a inteligência do projecto desde a selecção e limpeza morfológica do *corpus* textual até à mineração de dados por redes neuronais profundas e à sua consequente interpretação estatística combinada. O ciclo de vida dos dados foi mapeado estritamente através de cinco etapas estruturais:

1. **Selecção dos dados:** Isolamento e extracção de amostras textuais em inglês a partir dos repositórios científicos *HC3* e *OpenTuringBench*, garantindo a uniformidade e paridade temática das instâncias.
2. **Pré-processamento (*Data Pre-processing*):** Fase de higienização do ambiente, na qual se remove o ruído sintático, limpam-se dados inconsistentes e eliminam-se elementos supérfluos através da aplicação de expressões regulares para apagar pontuações e normalizar caracteres em caixa baixa.
3. **Transformação (*Transformation / Feature Engineering*):** Os dados limpos são convertidos em formatos matemáticos assimiláveis pelos modelos. Em processamento de linguagem natural (NLP), isto envolve a segmentação morfológica (*tokenization*) e a criação de matrizes estatísticas esparsas baseadas em pesos *TF-IDF* para os modelos tabulares, bem como a indexação densa de tokens para a alimentação de camadas de *embeddings*.
4. **Mineração de Dados (*Data Mining*):** estamos a falar do coração do processo, onde os algoritmos e redes neuronais são efectivamente aplicados para mapear padrões ocultos de forma automatizada. Nesta fase ocorrem os treinos competitivos da Regressão Logística Baseline, da *Deep Neural Network* (DNN) com tratamento de *overfitting* via *Dropout* em NumPy, e do classificador recorrente *Bi-LSTM* em PyTorch.
5. **Avaliação e Interpretação:** Analisam-se e comparam-se os resultados preditivos gerados pelos diferentes modelos através de métricas de performance combinadas (*Accuracy, Recall, Precision* e *F1-Score*), assegurando uma validação não-enviesada que qualifique os melhores classificadores para os rankings de submissão externa.

2. Concepção e Implementação dos Modelos

2.1. Processamento de Dados e Engenharia de Atributos (Tarefa 1)

O dataset de treino e validação resultou da fusão entre as amostras disponibilizadas pelos docentes e pares de respostas extraídos do dataset externo *Hello-SimpleAI/HC3*. Cada instância do corpus foi submetida a uma etapa de padronização, onde se aplicou a regra de conversão para minúsculas e a filtragem de caracteres não-alfabéticos via a expressão regular **r'\b[a-z]{2,}\b'**.

Para os modelos de baixo nível em NumPy, os textos foram transformados numa matriz tabular $\mathbf{X} \in \mathbf{R}^{NxD}$ representando as **D** = 1000 palavras mais relevantes segundo a métrica ***TF-IDF***. A frequência do termo **TF**(t, d) e a frequência inversa do documento **IDF**(t) foram calculadas de raiz segundo a formulação:

$$TF(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}, \quad IDF(t) = \ln\left(\frac{N+1}{DF(t)+1}\right) + 1$$

$$TF\text{-}IDF(t,d,D) = TF(t,d) \times IDF(t)$$

Para o modelo PyTorch, construiu-se um vocabulário indexado com mapeamento de tokens para inteiros únicos, aplicando-se *zero-padding* de modo a fixar a dimensão temporal máxima das sequências em **T** = 128 tokens.

2.2. *Modelos de Raiz em NumPy (Tarefa 2)*

Em conformidade com os requisitos do projecto, foi desenvolvida uma infraestrutura de modelação a partir do zero utilizando exclusivamente operações matriciais da biblioteca NumPy, sem recurso a bibliotecas de aprendizagem automática de alto nível.

2.2.1. *Baseline: Regressão Logística*

A Regressão Logística foi implementada para servir de *baseline* de comparação. Dado um vector de atributos $\mathbf{x_i} \in \mathbf{R^D}$, a probabilidade estimada de o texto ser gerado por IA ($\hat{y}_i = P(y=1|x_i)$) é dada pela função sigmoide:

$$\hat{y}_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}, \quad \text{onde } z_i = w^T x_i + b$$

O vector de pesos $\mathbf{w} \in \mathbf{R^D}$ e o viés $\mathbf{b} \in \mathbf{R}$ são optimizados através da minimização da função de custo de Entropia Cruzada Binária (*Binary Cross-Entropy Loss*):

$$\mathcal{L}(w, b) = -\frac{1}{N} \sum_{i=1}^{N} [y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)]$$

Os gradientes da perda em relação aos parâmetros são calculados por:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{N} X^T (\hat{Y} - Y), \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)$$

2.2.2. Deep Neural Network (DNN) com Dropout

Para capturar relações não-lineares nas matrizes tabulares *TF-IDF*, implementou-se uma *Deep Neural Network* (DNN) multicamadas em NumPy composta por uma camada de entrada (D=1000), duas camadas ocultas (h_1 = 64 e h_2 = 32 neurónios) e uma camada de saída sigmoidal (h_3 = 1).

- **Forward Pass e Funções de Activação:** Nas camadas ocultas $l \in \{1, 2\}$, utilizou-se a função de activação *ReLU* (*Rectified Linear Unit*):

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}, \quad A^{[l]} = \max(0, Z^{[l]})$$

Para mitigar o *overfitting*, incorporou-se a técnica de **Inverted Dropout** nas camadas ocultas durante a fase de treino. Para uma taxa de retenção p = 1 - dropout_rate, cria-se uma máscara binária de Bernoulli $M^{[l]} \sim$ Bernoulli(p):

$$\tilde{A}^{[l]} = \frac{A^{[l]} * M^{[l]}}{p}$$

- **Backward Pass (Backpropagation):** Os gradientes da função de perda foram propagados para trás utilizando a regra da cadeia e a derivada analítica da função *ReLU*:

$$\frac{\partial \mathcal{L}}{\partial Z^{[l]}} = \frac{\partial \mathcal{L}}{\partial A^{[l]}} * g'(Z^{[l]}), \quad \text{onde } g'(z) = \begin{cases} 1 & \text{se } z > 0 \\ 0 & \text{caso contrário} \end{cases}$$

2.3. Modelos Recorrentes em PyTorch (Tarefa 3)

Enquanto os modelos tabulares tratam o texto como uma colecção não-ordenada de palavras (*bag-of-words*), o modelo desenvolvido em PyTorch explora a dependência sequencial e a sintaxe da linguagem natural através de uma rede **Bi-LSTM (Bidirectional Long Short-Term Memory)**.

Texto (Sequência) → [Embedding Layer] →[Bi-LSTM Layer] →[Dropout] →[FC Layer] →Sigmoide

1. **Camada de Embedding:** Cada token da sequência de entrada é projectado num espaço vetorial denso e contínuo E Є R^{Vxd_e}, onde V = 10000 é a dimensão do vocabulário e d_e = 128 é a dimensão do *embedding*.
2. **Camada Bi-LSTM:** A rede processa a sequência de *embeddings* em ambas as direções simultaneamente (esquerda → direita e direita → esquerda). As portas da unidade LSTM (*forget*, *input* e *output*) controlam o fluxo de informação ao longo do tempo **t**:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \quad f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

$$c_t = f_t * c_{t-1} + i_t * \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \quad h_t = o_t * \tanh(c_t)$$

Ao concatenar os estados ocultos finais da passagem direta (h_T) e da passagem reversa (h_1), obtém-se uma representação global do contexto textual com dimensão 2 x d_h (2 x 128 = 256).

3. **Classificação:** A representação concatenada passa por uma camada linear totalmente conectada (*Fully Connected*) que produz o *logit* de saída, o qual é posteriormente convertido em probabilidade via função Sigmoide. O modelo foi otimizado utilizando o algoritmo *Adam* com uma taxa de aprendizagem $\eta = 0.001$ e minimizando a perda *BCEWithLogitsLoss*.

3. Avaliação Experimental e Resultados (Tarefa 4)

A avaliação dos modelos foi realizada comparando o desempenho da baseline de Regressão Logística em NumPy, da DNN em NumPy e da rede Bi-LSTM em PyTorch no conjunto de teste retido (10% do dataset total). Foram calculadas as métricas padrão de classificação binária:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad \text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

3.1. Tabela Comparativa de Desempenho

Modelo	Framework / Implementação	Precision	Recall	F1-Score	Accuracy
Regressão Logística (Baseline)	NumPy (de raiz)	0.8412	0.8250	0.8330	0.8350
Deep Neural Network (DNN)	NumPy (de raiz com Dropout)	0.8875	0.8710	0.8792	0.8800
Bi-LSTM Recorrente	PyTorch (com Embeddings)	0.9340	0.9215	0.9277	0.9290

3.2. Discussão dos Resultados

A análise comparativa permite extrair conclusões significativas sobre o problema de deteção de texto sintético:

- A **Regressão Logística (Baseline)** em NumPy estabeleceu um limiar de acuidade sólido de 83.5%, demonstrando que a frequência de certas palavras-chave (*TF-IDF*) oferece pistas iniciais sobre o autor (humano vs. IA).
- A **DNN em NumPy** superou a baseline em cerca de 4.5 pontos percentuais no *F1-Score*, provando que as combinações não-lineares capturadas pelas camadas ocultas e estabilizadas pelo Dropout trazem ganhos expressivos no tratamento de matrizes esparsas.
- O modelo **Bi-LSTM em PyTorch** registou o melhor desempenho global (F1-Score de 92.77% e Accuracy de 92.90%). A superioridade da Bi-LSTM justifica-se pelo facto de os modelos de linguagem generativos (LLMs) deixarem "pegadas digitais"

na estrutura sintática, na coerência sequencial e na ordem das palavras — padrões esses que são perdidos em abordagens baseadas em sacos de palavras (*Bag-of-Words*), mas capturados eficientemente por redes recorrentes bidirecionais.

Conclusão

Esta investigação permitiu conceber, implementar e avaliar com sucesso uma solução computacional robusta para a classificação binária automatizada entre textos em língua inglesa redigidos por seres humanos e conteúdos sintéticos gerados por Grandes Modelos de Linguagem (LLMs). O cumprimento do objectivo geral foi suportado pelo alcance sistemático de cada um dos objectivos específicos traçados no âmbito da metodologia KDD:

1. **Fundamentação e Estruturação do Projecto:** A revisão bibliográfica estabeleceu a base teórica sobre a representação vectorial de linguagem natural e o funcionamento de redes profundas. A criação e organização do repositório garantiram a reprodutibilidade e modularidade de todo o ciclo de desenvolvimento.
2. **Engenharia de Dados e Multi-Fonte:** A compilação, fusão e higienização das amostras do corpus disponibilizado pelos docentes com o *dataset* externo *HC3* permitiram construir uma base equilibrada e representativa. O pré-processamento via expressões regulares e a extracção de matrizes *TF-IDF* eliminaram ruídos sintáticos e isolaram as propriedades estatísticas mais relevantes do texto.
3. **Modelação de Baixo Nível (NumPy):** A concepção do ciclo de treino do zero — sem recurso a bibliotecas de alto nível — validou a formulação matemática da Regressão Logística (*baseline*) e da *Deep Neural Network* (DNN). Os testes comprovaram que a introdução de camadas não-lineares (*ReLU*) e a regularização por *Inverted Dropout* na DNN aumentaram a capacidade de generalização do modelo tabular, elevando o *F1-Score* de $83,30\%$ para $87,92\%$.
4. **Modelação Recorrente Avançada (PyTorch):** A implementação da rede *Bi-LSTM* com camadas de *embeddings* e optimização de hiperparâmetros (optimizador *Adam* e *dropout*) permitiu capturar a dependência sequencial e a sintaxe temporal do texto. Este modelo alcançou o desempenho superior na investigação, registando uma *Accuracy* de $92,90\%$ e um *F1-Score* de $92,77\%$.
5. **Pipelines de Submissão e Validação Externa:** A estruturação do pipeline de teste automatizado garantiu a correta inferência de amostras não vistas e fundamentou o sucesso da 1ª Submissão dos modelos. A validação competitiva demonstrou que a preservação da ordem cronológica e sintática das palavras (*Bi-LSTM*) supera substancialmente a abordagem estatística não-ordenada (*bag-of-words/TF-IDF*), uma vez que os LLMs deixam "marcas estilísticas" na coesão sintática que apenas redes sequenciais conseguem mapear.

Em síntese, os resultados obtidos confirmam a hipótese de que a combinação entre pré-processamento rigoroso e arquitecturas profundas recorrentes é altamente eficaz para auditar a autenticidade de documentos digitais. Como trabalhos futuros, recomenda-se a expansão desta infraestrutura para cenários de *fine-tuning* em arquitecturas baseadas em *Transformers* (e.g., RoBERTa ou DistilBERT), bem como a avaliação do comportamento dos classificadores perante estratégias de ataque adversarial e paráfrase automática.

Referências Bibliográficas

1. Rocha, M., & Makili, L. (2024). *Unidade Curricular de Aprendizagem Profunda: Enunciado do Trabalho Prático em Grupo*. Instituto Politécnico da Universidade Katyavala Bwila.
2. Guo, B., Zhang, X., Wang, Z., Jiang, M., Nie, J., Ding, Y., Yue, J., & Ma, Y. (2023). *How Close is ChatGPT to Human Experts? Comparison Corpus (HC3)*. arXiv preprint arXiv:2301.07597.
3. Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). *From data mining to knowledge discovery in databases*. AI magazine, 17(3), 37-37.
4. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
5. Hochreiter, S., & Schmidhuber, J. (1997). *Long short-term memory*. Neural computation, 9(8), 1735-1780.